

Domine a **Infraestrutura** como Código

Ansible, Terraform, Puppet e Vagrant: Do caos manual à automação previsível.

O Caos da **Deriva de Configuração**

Analogia: A Receita de Bolo Alterada

Imagine que vários chefs adicionam um ingrediente secreto na receita ao longo dos anos. No final, o bolo original não existe mais.

- ▶ **Configuration Drift:** Ocorre quando servidores que deveriam ser idênticos divergem devido a intervenções manuais.
- ▶ **O Objetivo da GC:** Consistência, repetibilidade e integridade em todo o ciclo de vida.
- ▶ **Solução:** Substituir scripts manuais ad-hoc por definições declarativas automatizadas.



Pilares da Gerência de Configuração



Idempotência

Analogia do Elevador: Apertar o botão do "5º andar" dez vezes ainda leva você apenas ao 5º andar. Operações executadas várias vezes resultam sempre no mesmo estado final, sem ações redundantes.



Estado Desejado

Analogia do GPS: O sistema foca no destino, não nos passos para chegar lá. A ferramenta compara o estado atual com o declarado e aplica apenas as mudanças necessárias.



Inventário

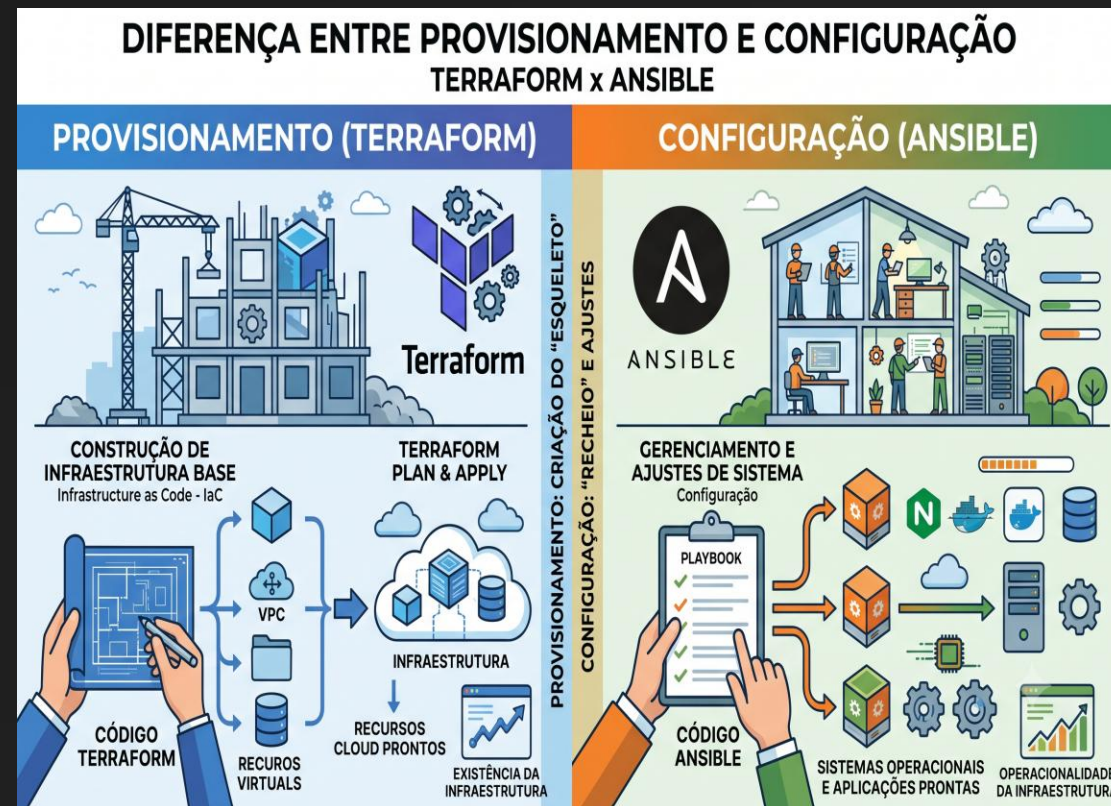
A Lista de Contatos: Uma organização lógica dos seus ativos (webserver, databases). Permite direcionar políticas específicas para o conjunto correto de servidores.

Provisionamento vs. Configuração

Camadas da Automação

Embora se complementem, existem distinções cruciais entre preparar o "terreno" e configurar o "interior" da infraestrutura.

- ▶ **Terraform (Provisionamento):** Foca na criação da infraestrutura em si. Ex: Criar 3 máquinas virtuais na AWS.
- ▶ **Ansible (Configuração):** Foca no software que rodará sobre a infraestrutura. Ex: Instalar o Nginx nessas máquinas.



Terraform: O **Arquiteto** (IaC)

Construindo a Fundação

Analogia: A equipe que ergue as paredes, instala a rede elétrica e os canos da casa. O Terraform garante que a planta seja seguida à risca.

- ▶ **Agnóstico:** Gerencia recursos em diversas nuvens (AWS, Azure, GCP).
- ▶ **Declarativo:** Usa a linguagem HCL para descrever os recursos desejados.
- ▶ **Providers e Grafos:** Constrói um mapa de dependências lógicas para criar e destruir na ordem correta.



Vagrant: Fim do "Na Minha Máquina Funciona"



Padronização Local

Garante que o ambiente do desenvolvedor seja idêntico ao da produção usando um simples arquivo `Vagrantfile`.



Boxes & Providers

Utiliza "Boxes" (imagens pré-configuradas) e integra-se de forma transparente a virtualizadores como VirtualBox e Docker.



Automação (Provisioning)

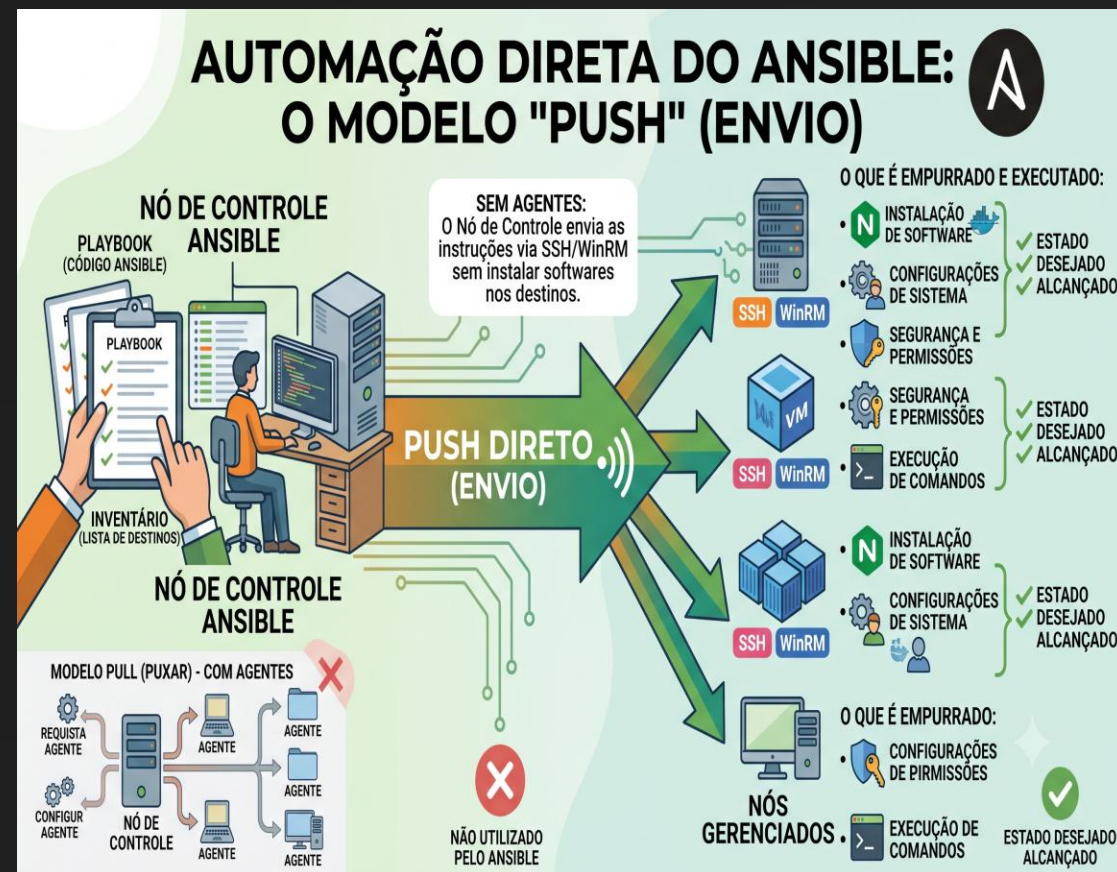
O Vagrant chama automaticamente Ansible ou Puppet durante o boot para instalar softwares e preparar o setup.

Ansible: Automação **Direta** (Push)

O Comandante via Rádio

Analogia: Como um general que envia ordens precisas por rádio. Os soldados (servidores) executam imediatamente, sem precisar instalar um software receptor especial.

- ▶ **Agentless:** Sem necessidade de agentes. Usa protocolos padrão (SSH / WinRM).
- ▶ **Modelo Push:** O nó controlador envia diretamente as configurações.
- ▶ **Playbooks e Módulos:** Arquivos YAML legíveis que agrupam módulos reutilizáveis de instalação e configuração.



Puppet: O Guardiã do Estado (Pull)

O Inspetor Residente

Analogia: Um segurança interno em cada servidor que consulta periodicamente o manual de regras central e ajusta tudo o que saiu do lugar.

- ▶ **Modelo Pull (Agente/Servidor):** Agentes em cada nó puxam e aplicam regras do Puppet Server.
- ▶ **Manifestos & Catálogo:** Usa DSL baseada em Ruby. O Server compila os manifestos em um Catálogo local.
- ▶ **Correção Contínua:** O agente roda frequentemente (ex: a cada 30 min) para reverter desvios de configuração.



Dúvidas?

Prontos para aplicar infraestrutura como código?

Ciclo de Vida de Software & Automação | Infraestrutura DevOps
